boruvkas

```python
def alg(n, edges):
    m = dict()
    p = list(range(n))
    mst = []
    def f(x):
        if p[x] != x: p[x] = f(p[x])  # path compression
        return p[x]
    while len(mst) < n - 1:
        m.clear()
        for u, v, w in edges:
            ru, rv = map(f, (u, v))
            if ru != rv:
                if ru not in m or w < m[ru][2]: m[ru] = (u, v, w)
                if rv not in m or w < m[rv][2]: m[rv] = (u, v, w)
        for e in m.values():
            if f(e[0]) != f(e[1]):
                mst.append(e)
                p[f(e[0])] = f(e[1])
    return mst

def main():
    inp = ""
    with open("../test_data/graph_input.txt", encoding="utf-8") as f:
        inp = f.read()
    outp = ""
    with open("../test_data/mst_output.txt", encoding="utf-8") as f:
        outp = f.read()
    edges = [tuple(map(int, line.split(','))) for line in inp.strip().split('\n')]
    expected = [tuple(map(int, line.split(','))) for line in outp.strip().split('\n')]
    n = max(max(u, v) for u, v, w in edges) + 1
    result = alg(n, edges)
    result_pairs = [(u, v) for u, v, w in result]
    result_set = set((min(u,v), max(u,v)) for u, v in result_pairs)
    expected_set = set((min(u,v), max(u,v)) for u, v in expected)
    print("PASSED" if result_set == expected_set else "FAILED")

if __name__ == "__main__":
    main()
```

gaussian_elimination

```python
# https://www.geeksforgeeks.org/dsa/gaussian-elimination/
MAXN = 100

def partial_pivot(A, n):
    for i in range(n):
        pivot_row = i
        for j in range(i + 1, n):
            if abs(A[j][i]) > abs(A[pivot_row][i]):
                pivot_row = j
        if pivot_row != i:
            A[i], A[pivot_row] = A[pivot_row], A[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n + 1):
                A[j][k] -= factor * A[i][k]

def back_substitute(A, n):
    x = [0]*n
    for i in reversed(range(n)):
        sum_val = sum(A[i][j] * x[j] for j in range(i + 1, n))
        x[i] = (A[i][n] - sum_val) / A[i][i]
    return x

if __name__ == "__main__":
    n = 3
    A = [[3.0, 2.0, -4.0, 3.0],
         [2.0, 3.0, 3.0, 15.0],
         [5.0, -3, 1.0, 14.0]]
    partial_pivot(A, n)
    x = back_substitute(A, n)
    print("Solution for the system:")
    for i in range(n):
        print(f"{x[i]:.6f}")

# 3.000000
# 1.000000
# 2.000000
```

goodbye

```
print("goodbye world")
```

hello

```
print("hello world")
```